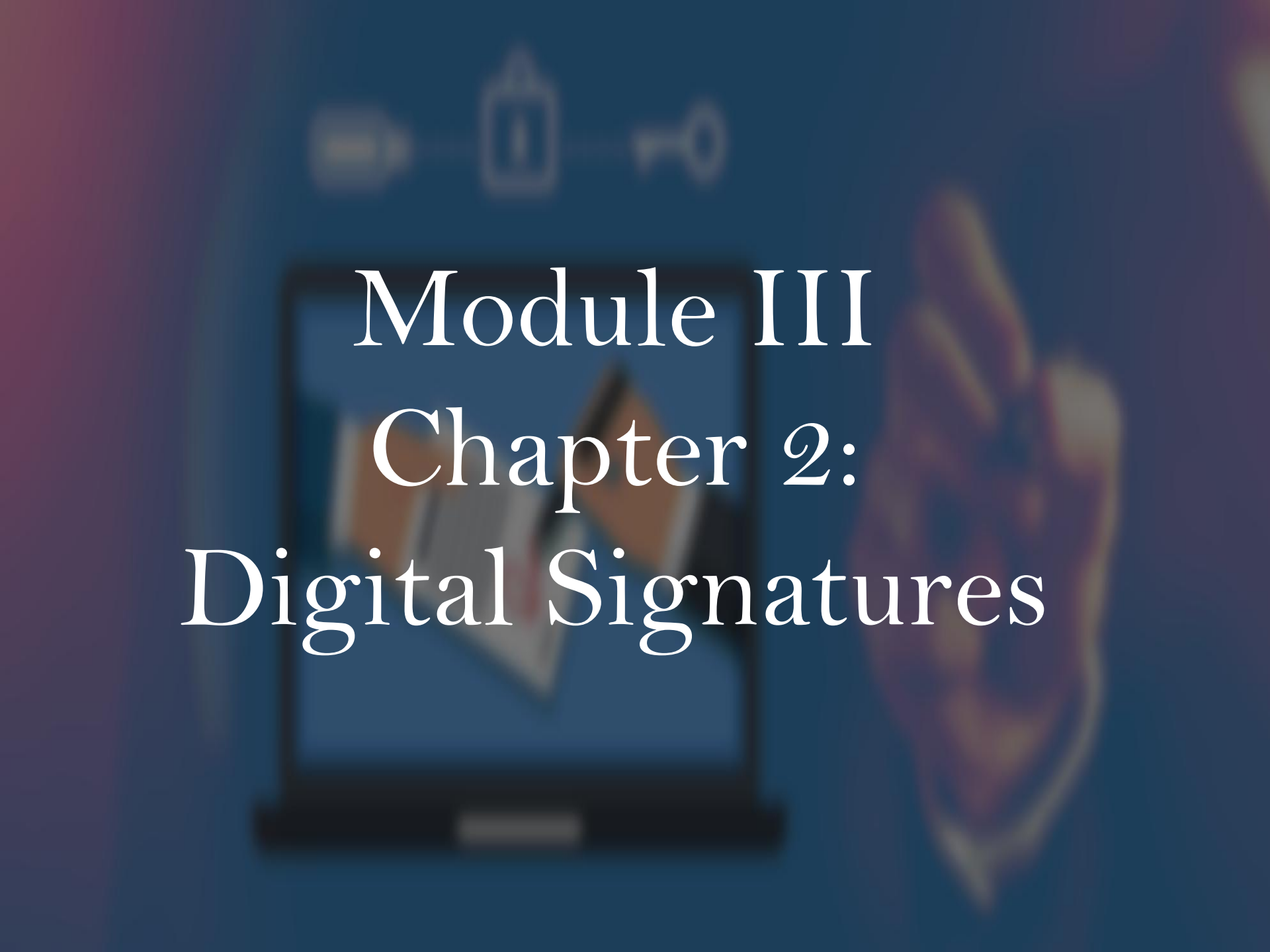




Module III

Chapter 2:

Digital Signatures

What is a digital signature?

- A digital signature is a mathematical technique used to **validate the authenticity and integrity of a digital document, message or software.**
- It's the **digital equivalent of a handwritten signature or stamped seal**, but it offers far more inherent **security.**
- A digital signature is intended to **solve the problem of tampering and impersonation in digital communications.**
- Digital signatures can provide **evidence of origin, identity and status of electronic documents, transactions and digital messages.**
- Signers can also use them to **acknowledge informed consent.**
- In many countries, including the U.S., digital signatures are considered legally binding in the same way as traditional handwritten document signatures.

How do digital signatures work?

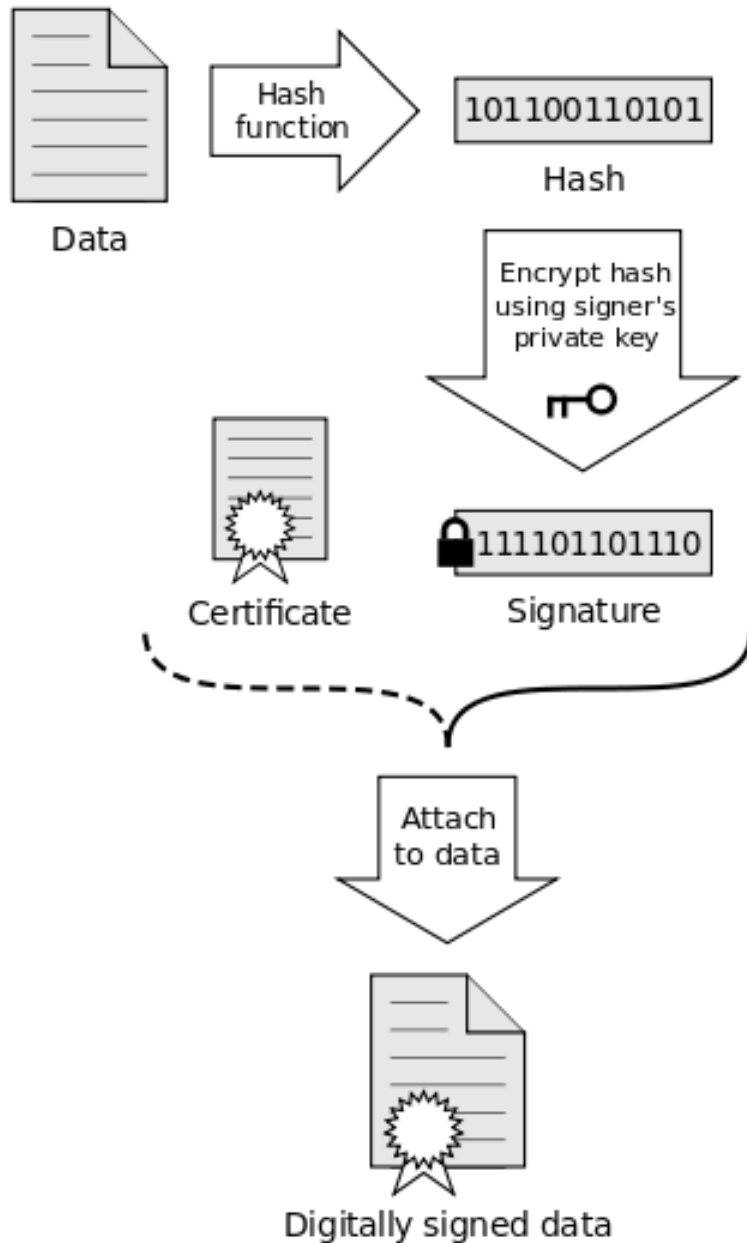
- Digital signatures are **based on public key cryptography**, also known as **asymmetric cryptography**.
- Using a public key algorithm, such as **RSA**, two keys are generated, creating a mathematically linked pair of keys: **one private and one public**.
- For encryption and decryption, the person who **creates the digital signature uses a private key to encrypt signature-related data**. The only way to decrypt that data is with the **signer's public key**.
- If the recipient can't open the document with the signer's public key, that indicates there's a problem with the document or the signature. This is how digital signatures are authenticated.

- Digital signing certificates, also called **public key certificates**, are used to verify that the public key belongs to the issuer.
- **Signing certificates are sent with the public key; they contain information about the certificate's owner, expiration dates and the digital signature of the certificate's issuer.**
- **Trusted third-party certificate authorities (CAs), such as DocuSign or GlobalSign, issue signing certificates.**
- If someone else has access to the private signing key, that party could create fraudulent digital signatures in the name of the private key holder.

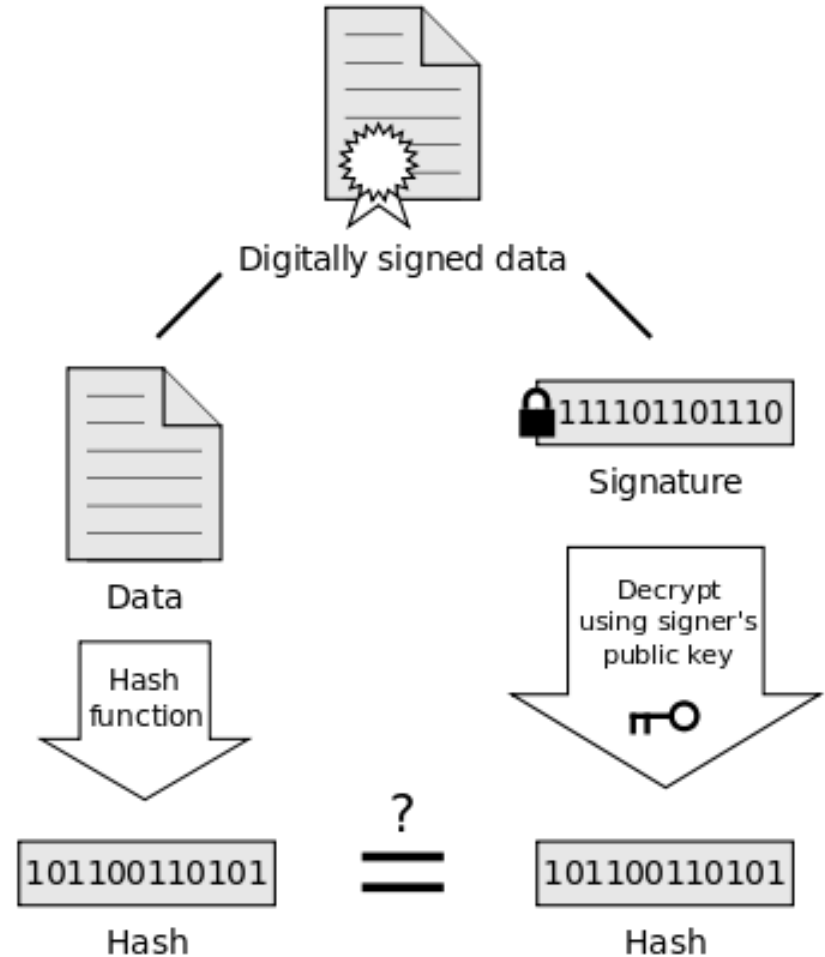
The steps followed in creating a digital signature are:

- **Message digest is computed by applying the hash function on the message and then message digest is encrypted using the private key of the sender to form the digital signature.**
 - **(digital signature = encryption (private key of sender, message digest) and message digest = message digest algorithm (message)).**
- A digital signature is then transmitted with the message.
 - **(message + digital signature is transmitted)**
- The receiver **decrypts the digital signature using the public key of the sender.**
- The **receiver now has the message digest.**
- The **receiver can compute the message digest from the message**
 - (actual message is sent with the digital signature).
- The message digest computed by receiver and the message digest
 - (got by decryption on digital signature) need to be same for ensuring integrity.

Signing



Verification



If the hashes are equal, the signature is valid.

Assurances About Digital Signatures:

- **Authenticity:** The identity of the signer is verified.
- **Integration:** Since the content was digitally signed, it hasn't been altered or interfered with.
- **Non-repudiation:** demonstrates the source of the signed content to all parties. The act of a signer denying any affiliation with the signed material is known as repudiation.
- **Notarization:** Under some conditions, a signature in a Microsoft Word, Microsoft Excel, or Microsoft PowerPoint document that has been time-stamped by a secure time-stamp server is equivalent to a notarization.

Benefits of Digital Signatures

- **Legal documents and contracts:** Digital signatures are **legally binding**. This makes them ideal for any legal document that requires a signature authenticated by one or more parties and guarantees that the record has not been altered.
- **Sales contracts:** Digital signing of contracts and sales contracts authenticates the **identity of the seller and the buyer**, and both parties can be sure that the signatures are legally binding and that the terms of the agreement have not been changed.
- **Financial Documents:** Finance departments digitally sign invoices so **customers can trust that the payment request is from the right seller**, not from an attacker trying to trick the buyer into sending payments to a fraudulent account.
- **Health Data:** In the healthcare industry, privacy is paramount for both patient records and research data. Digital signatures ensure that this **confidential information was not modified when it was transmitted between the consenting parties**.

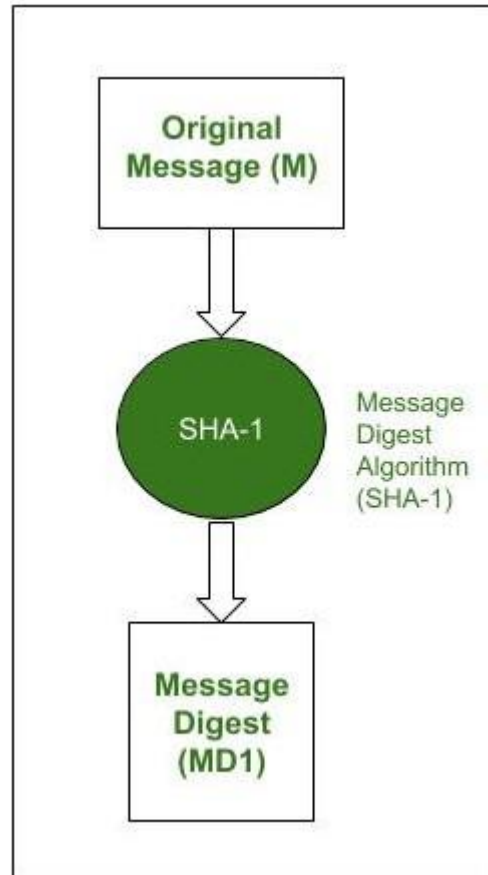
Drawbacks of Digital Signature

- **Dependency on technology:** Because digital signatures **rely on technology**, they are **susceptible to crimes, including hacking**. As a result, businesses that use digital signatures must **make sure their systems are safe and have the most recent security patches and upgrades installed**.
- **Complexity:** Setting up and using digital signatures can be challenging, especially for **those who are unfamiliar with the technology**. This **may result in blunders and errors that reduce the system's efficacy**. The process of issuing digital signatures to **senior citizens can occasionally be challenging**.
- **Limited acceptance:** Digital signatures take **time to replace manual ones since technology is not widely available in India, a developing nation**.

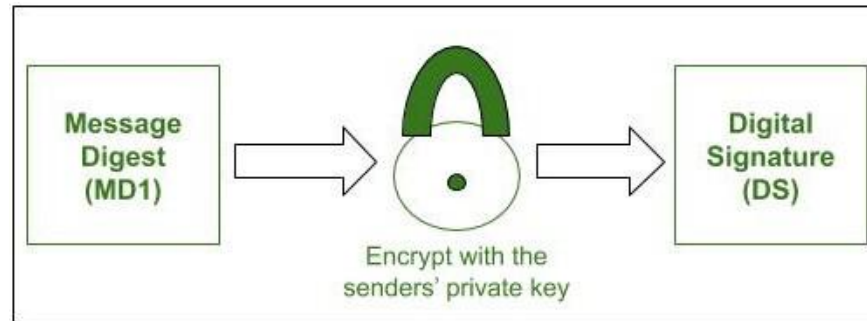
RSA Signature Scheme

- Assume that there is a sender (A) and a receiver (B).
- A wants to send a message (M) to B along with the digital signature (DS) calculated over the message.

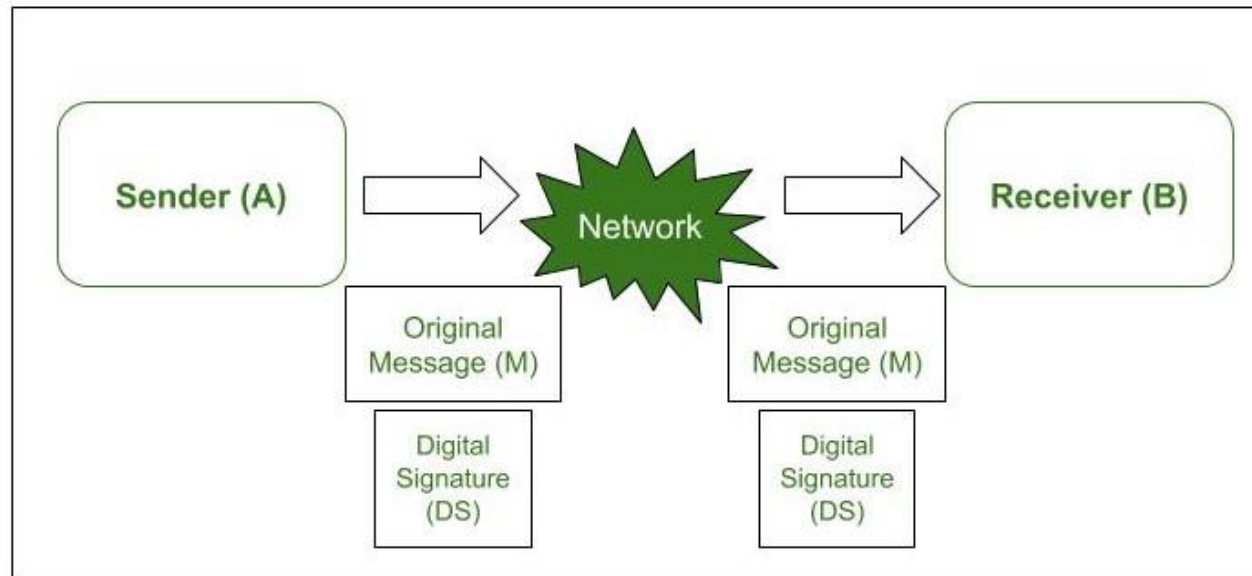
Step 1: Sender A uses the SHA-1 Message Digest Algorithm to calculate the message digest (MD1) over the original message M.



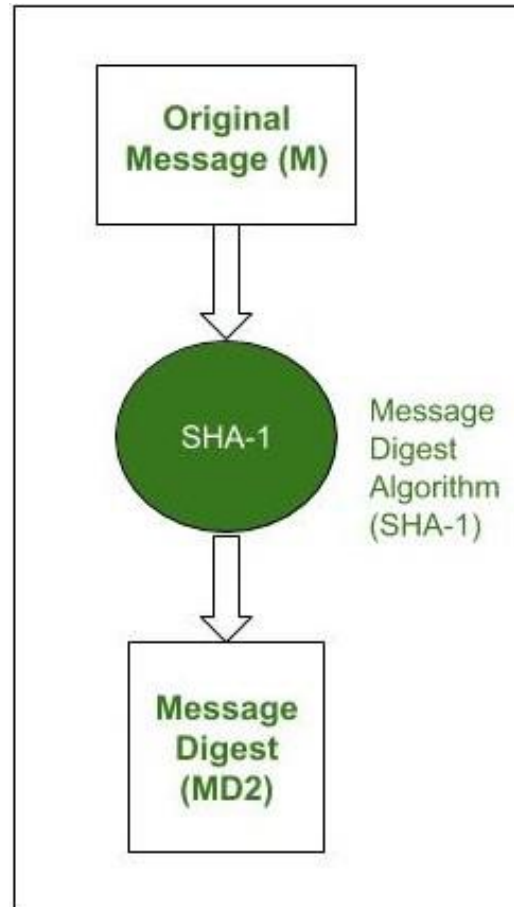
Step 2 : A now encrypts the message digest with its private key. The output of this process is called Digital Signature (DS) of A.



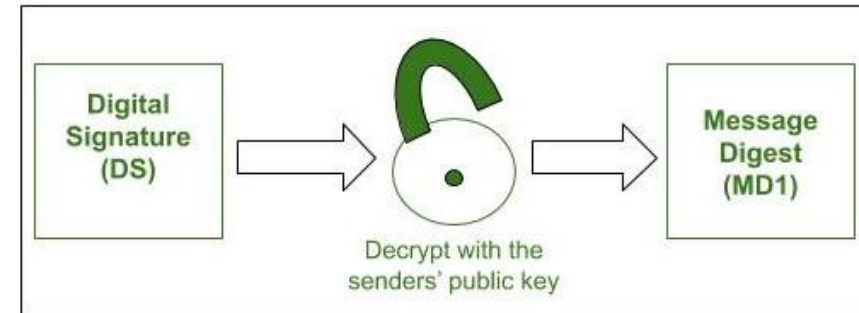
Step 3 : Now sender A sends the digital signature (DS) along with the original message (M) to B.



Step 4 : When B receives the Original Message(M) and the Digital Signature(DS) from A, it first uses the same message-digest algorithm as was used by A and calculates its own Message Digest (MD2) for M.

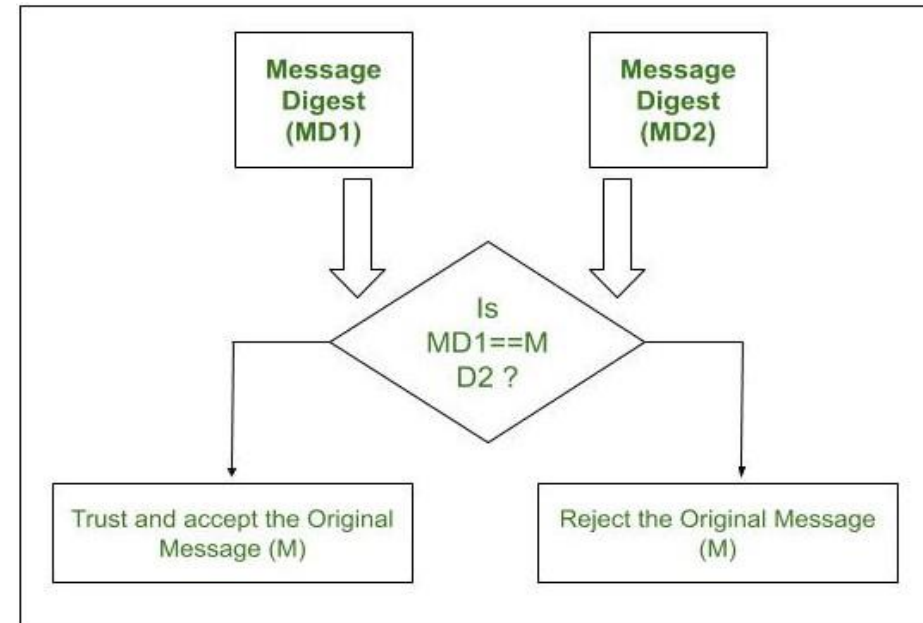


Step 5 : Now B uses A's public key to decrypt the digital signature because it was encrypted by A's private key. The result of this process is the original Message Digest (MD1) which was calculated by A.



Step-6 : If $MD1 == MD2$, the following facts are established as follows.

- B accepts the original message M as the correct, unaltered message from A.
- It also ensures that the message came from A and not someone posing as A.



- The message digest (MD1) was encrypted using A's private key to produce a digital signature. Therefore, the digital signature can be decrypted using A's public key (due to asymmetric form of RSA).
- If the receiver B is able to decrypt the digital signature using A's public key, it means that the message is received from A itself and now A cannot deny that he/she has not sent the message.
- It also proves that the original message did not tamper because when the receiver B tried to find its own message digest MD2, it matched with that of A's MD1.
- Suppose a malicious user tries to access the original message and perform some alteration.
- Now he/she will calculate a new message digest over the altered message. It might concern you with data integrity and confidentiality but here's the catch.
- The attacker will have to sign the altered message using A's private key in order to pose as A for the receiver B.
- However, an attacker cannot sign the message with A's private key because it is known to A only.
- Hence, the RSA signature is quite strong, secure, and reliable.

Attacks on RSA Digital Signature

1. Chosen-message Attack :

- In a **chosen-message attack**, the attacker tries to make the signer sign a message selected by the attacker.
The attacker's goal is to get a **valid digital signature** on a message that the signer never really wanted to sign.
- Let us use:
 - **A** = Alice, the real signer
 - **C** = Charlie, the attacker

A) Generic Chosen-message Attack

- In this attack, Charlie somehow tricks Alice into signing some messages **without Alice realizing what she is actually signing**.
- Charlie may not know Alice's public key in detail, but he manages to get her signature on attacker-chosen content.
- Alice thinks she is signing some harmless data, but actually she is signing something useful for Charlie.

Example

- Suppose Alice is a manager in a company.
- Charlie sends Alice a file and says: "Please sign this attendance confirmation document."
- But the hidden content is actually: "Approve payment of ₹50,000 to Charlie."
- Alice signs it without carefully checking.
- Now Charlie has Alice's valid digital signature on a message she **never intended to approve**.

B) Direct Chosen-message Attack

- In this attack, Charlie knows Alice's **public key** and cleverly prepares messages so that Alice's signature on one message can be reused or manipulated for another message.
- This is more direct and more technical.

Example:

- Suppose Alice signs only approved office notices.
- Charlie prepares one message: "Meeting at 10 AM"
- and gets Alice to sign it.
- Then Charlie modifies or substitutes the original message in such a way that the same signature appears attached to another message like: "Transfer project ownership to Charlie"
- Now it looks as if Alice signed the second message, even though she signed only the first one.
- **In simple words**
- Charlie:
 - knows how Alice's signature system works,
 - gets her to sign one message,
 - then changes or replaces the message,
 - keeps the signature unchanged,
 - and claims Alice signed the new message.

2. Key-only Attack :

- A **key-only attack** means the attacker knows **only the public key** of the signer and tries to create a fake valid signature.
- In RSA digital signature:
 - the **private key** is secret and used for signing
 - the **public key** is open to everyone and used for verification
- So in this attack, the attacker has only:
 - Alice's public key
 - no private key
 - no help from Alice
 - no signed messages to study
- **Simple meaning**
 - The attacker tries to break the signature system using only public information.

Example

Suppose:

- Alice publishes her public key so everyone can verify her signatures.
- Charlie wants to send a fake message: **“I approve this contract – Alice”**
- But Charlie does **not** have Alice’s private key.
- He only has Alice’s **public key**.
- Now Charlie tries to guess or mathematically create a signature that will pass verification with Alice’s public key.
- If the RSA system is strong and properly used, Charlie should **not** be able to do this.
- It is like trying to copy a person’s handwritten signature by only seeing their name printed in a directory, without ever seeing the real signature.
- A **key-only attack** is the **weakest attack model** because the attacker has very little information.

3. Known-message Attack :

- A **known-message attack** means the attacker has access to some **message and signature pairs** that were already signed by the real signer.
- That means the attacker knows:
 - the original message
 - the valid signature for that message
 - the public key of the signer
 - Then the attacker studies these signed messages and tries to create a **new fake valid signature** for another message.
- The attacker cannot choose the messages to be signed.
But the attacker can **observe** some messages that were already signed and use them to attack the system.

Example

- Suppose Alice is a company manager and she has already signed these messages:
 - **“Approved leave for Ravi”** → with Alice’s signature
 - **“Approved budget for lab”** → with Alice’s signature
- Charlie, the attacker, collects these message-signature pairs.
- Now Charlie studies them and tries to produce a valid signature for a new message like:
- **“Approved payment to Charlie”**
- Even though Alice never signed this message.
- This is called a **known-message attack** because Charlie is using **known signed messages** to try to forge another signature.
- It is like a person seeing many real handwritten signatures of Alice and then trying to imitate her signature on a new document.

Key idea

- In a known-message attack:
 - attacker does **not** ask the signer to sign new messages
 - attacker only uses **already available signed messages**
 - goal is to create a valid signature on a different message

Process of Creating a Digital Signature

- **Hashing:** The first step in creating a digital signature is to create a hash of the message or document that needs to be signed. This is done using a hash function, which produces a fixed-length output (the hash value) from an input of any size.
- **Signing:** The hash value is then encrypted using the private key of the signer. This produces the digital signature, which is attached to the original message or document.
- **Verification:** To verify the authenticity of the digital signature, the recipient of the message or document must first decrypt the signature using the public key of the signer. This produces the original hash value. The recipient then calculates the hash value of the received message or document using the same hash function that was used by the signer. If the two hash values match, the signature is valid and the message or document has not been tampered with.

Digital Signature Standard (DSS)

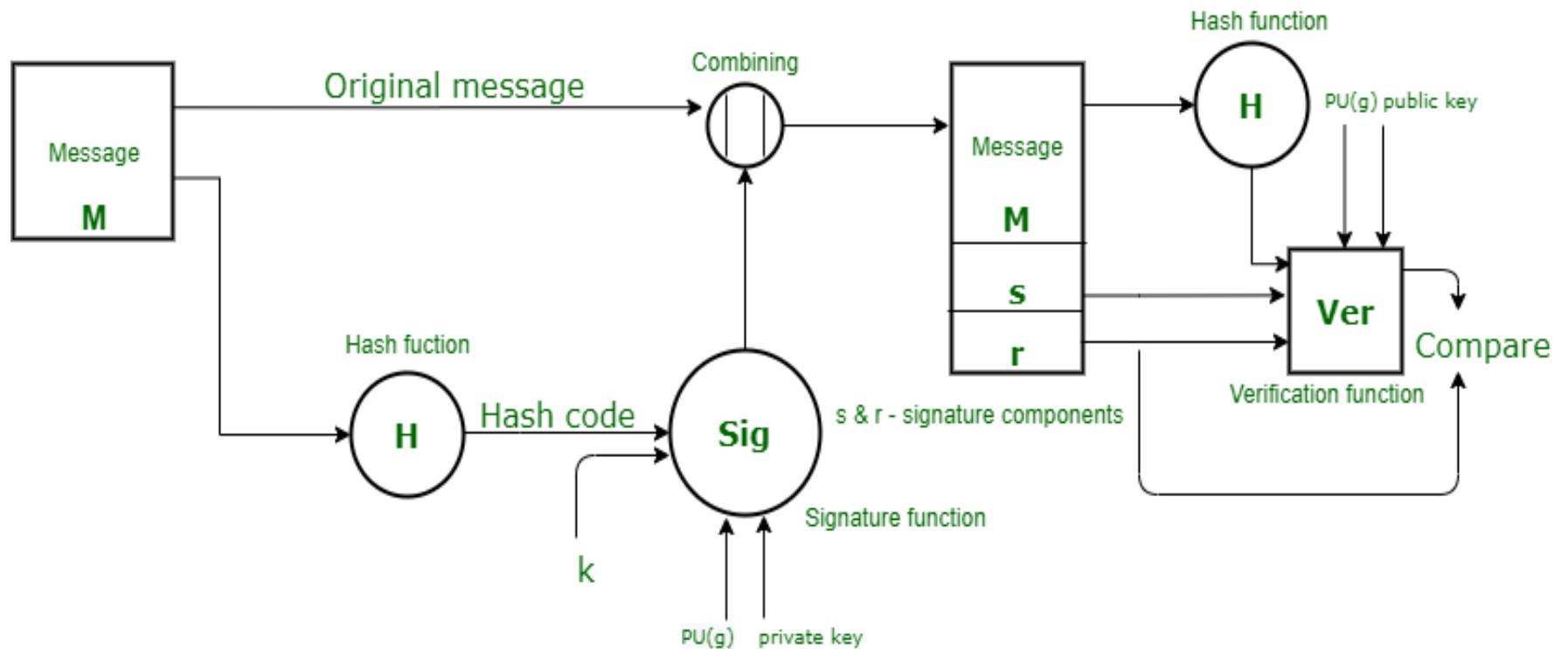
- Digital Signature Standards (DSS) are **specific algorithms used by applications that require a digital signature.**
- They're a **set of rules and parameters** that allow tracking of the signature to verify the identity of the signer.
- For a digital signature **to be authentic**, it must adhere to all **DSS regulations.**

How do Digital Signature Standards work?

- When a sender wants to digitally sign a document, they use their private key to generate a unique digital signature. Recipients can then use the sender's public key to verify the signature, helping to ensure the document's integrity and confirming its origin.
- Various DSS types employ different algorithms and security measures, offering varying levels of security and efficiency.

SENDER A

RECEIVER B



Sender Side: In DSS Approach, a hash code is generated out of the message and following inputs are given to the signature function –

- The hash code.
- The random number 'k' generated for that particular signature.
- The private key of the sender i.e., $PR(a)$.
- A global public key(which is a set of parameters for the communicating principles) i.e., $PU(g)$.

These input to the function will provide us with the output signature containing two components – 's' and 'r'. Therefore, the original message concatenated with the signature is sent to the receiver.

Receiver Side : At the receiver end, verification of the sender is done. The hash code of the sent message is generated. There is a verification function which takes the following inputs –

- The hash code generated by the receiver.
- Signature components 's' and 'r'.
- Public key of the sender.
- Global public key.

The output of the verification function is compared with the signature component 'r'. Both the values will match if the sent signature is valid because only the sender with the help of its private key can generate a valid signature.

3 types of digital signatures.

Although all electronic signatures must follow DSS rules, they're not all created equal. There are three different types of digital signatures you can use to sign documents:

- **Simple electronic signature (SES):**

This is the most **basic form of electronic signature**. SES are **quick and easy to add to documents**, but they're not protected by encryption methods. In other words, they're **not as secure**. Your **email signature** is a good example of an SES.

- **Advanced electronic signatures (AES):**

Although still not legally binding, AES do track changes to the document made after signing. This increases the security, but you still wouldn't want to use an AES on important contracts or documents.

- **Qualified advanced electronic signatures (QES):**

This is the **safest way to sign electronically**. Qualified electronic signatures, or when you create digital signatures, **use public key infrastructure, asymmetric cryptography**, and **two-factor authentication** to ensure the highest level of security. They can also **validate the signer's identity**, making them as safe and legal as a physical signature.

Benefits of advanced signature:

1. A digital signature provides strong security for transactions.
2. It prevents unauthorized users from forging documents.
3. Signed documents can be easily tracked.
4. It allows fast document delivery.
5. It is legally valid when issued by a government-authorized certifying authority.
6. A signer cannot deny signing the document.
7. Date and time are automatically stamped when the document is signed.
8. A digitally signed document cannot be copied or altered easily.
9. It confirms the identity of the signer.
10. It reduces the risk of fraud and impersonation.

Burdens of computerized signature:

1. Compatibility issues may arise because of different driver and software versions.
2. Software support is a major concern when using a digital signature certificate.
3. Businesses such as import-export firms may need digital signatures for e-tagging.
4. Risks include loss or theft of keys and weak storage systems.
5. Common standards are needed so different systems can work together.
6. Many technology products become outdated quickly.
7. Both sender and receiver may need to buy digital certificates.
8. Verification software may also need to be purchased.
9. Using digital signatures can involve extra cost for businesses.

One-Time Signature Scheme in Cryptography

- A One-Time Signature (OTS) scheme is a cryptographic method designed to provide **digital signatures for a single message or transaction**.
- This system ensures that each **signature can only be used once**, making it inherently secure for its intended purpose.

Definition:

- A One-Time Signature scheme is a cryptographic algorithm that **allows a user to sign only one message securely with a single key pair**.
- The private key used to sign a message **cannot be reused to sign any other message**, ensuring the security and authenticity of the signed message.

Key Features:

- Single Usage of Key Pair
- Security
- Efficiency
- Unforgeability

Advantages:

- High Security
- Simplicity
- Low Computational Overhead

Disadvantages:

- **Limited Use:** This makes it impractical for applications where multiple messages need to be signed over time, such as secure email or blockchain transactions.
- **Key Management:** Since each private key can only be used once, managing a large number of key pairs for multiple messages can become cumbersome and inefficient.
- **Scalability Issues:** If many messages need to be signed, it may require generating and managing an excessive number of key pairs, which is impractical for most real-world applications.

Applications:

- Cryptographic Protocols
- Blockchain

I. Lamport Signature Scheme

Basic Components

- **Private Key:** A set of randomly generated pairs of numbers.
- **Public Key:** A set of hash values derived from the private key.
- **Hash Function:** A cryptographic hash function (like SHA-256) that's easy to compute but hard to reverse.

1. Key Generation:

- You create a bunch of **secret numbers**.
- For every bit (0 or 1), you create **2 secrets**
- So for 1 position:
 - One secret for **0**
 - One secret for **1**
- If your message has 4 bits → you need:
 - $4 \times 2 = \mathbf{8 \text{ secrets}}$

Example:

Position	Secret for 0	Secret for 1
1	A	B
2	C	D
3	E	F
4	G	H

These are your **PRIVATE KEY**

Step 2: Create Public Key

- Each secret → convert into a hash
- Example:
 - hash(A), hash(B), hash(C), etc.
- These hashes become your **PUBLIC KEY**

Position	hash(0-secret)	hash(1-secret)
1	hash(A)	hash(B)
2	hash(C)	hash(D)
3	hash(E)	hash(F)
4	hash(G)	hash(H)

Step 3: Signing a Message

- Let's say your message becomes: **1010**
- Now for each bit:
 - If bit = **1** → pick secret for 1
 - If bit = **0** → pick secret for 0

• So:

Bit	Use Secret
1	B
0	C
1	F
0	G

Step 4: Verification

- Takes your signature (B, C, F, G)
- Hashes each one
- Compares with your public key
- If everything matches - Message is real and unchanged

Receiver has:

- **Public Key** → only hash values
- **Signature** → the secrets you revealed
- **Message** → e.g., 1010

Receiver checks **bit by bit**

Bit 1 = 1

You gave: B

Receiver computes: $\text{hash}(B)$

Compares with public key:

Should match: $\text{hash}(B)$ (column for bit=1)

Bit 2 = 0

You gave: C

Friend computes: $\text{hash}(C)$

Compares with:

Should match: $\text{hash}(C)$ (column for bit=0)

Bit 3 = 1

You gave: F

Check:

$\text{hash}(F) == \text{public key's hash}(F)$

Bit 4 = 0

You gave: G

Check:

$\text{hash}(G) == \text{public key's hash}(G)$

Advantages of Lamport Signature

- High Security
- Simplicity
- Resistance to Quantum Attacks: The Lamport signature scheme is resistant to quantum attacks, as it does not rely on problems like integer factorization or elliptic curve discrete logarithms, which are vulnerable to quantum algorithms.

Disadvantages of Lamport Signature

- Single Use
- Large Signatures
- Public Key Size

Practical Applications

- Post-Quantum Cryptography
- One-Time Password (OTP) Systems
- Cryptographic Research

II. WOTS

- WOTS stands for "**Winternitz One-Time Signature.**"
- It's a variation of the Lamport one-time signature scheme, designed to reduce the size of the signature.
- **The Problem with Lamport:**
 - Lamport signatures produce very large signatures, especially for signing long messages. This is because you need to reveal a portion of your private key for each bit of the message.
- **WOTS Solution:**
 - WOTS addresses this by grouping bits of the message together and applying a hash function multiple times.
 - This reduces the number of private key values that need to be revealed, resulting in a smaller signature.

Key Concepts

- **Hash Function:** WOTS uses a cryptographic hash function (e.g., SHA-256) to generate secure signatures.
- **Private Key:** A secret key used for signing the message.
- **Public Key:** A key that can be used to verify the signature.
- **One-Time:** The private key is only valid for signing one message. After it's used to sign a message, it should not be reused.

How WOTS Works

Step 1: Choose a Base (w)

- This is called the **Winternitz parameter**
- Example:
 - $w = 4$ (means values from 0 to 3)
 - $w = 16$ (more efficient, commonly used)
- This groups bits together

Example

- Message (binary): 10101100
- If $w = 4 \rightarrow$ group into 2-bit chunks: 10 10 11 00
- Convert each to decimal:

Binary	Decimal
10	2
10	2
11	3
00	0

Step 2: Secret Keys

- Instead of 2 secrets per bit (Lamport),
- We create **one secret per chunk**
- Example:

Chunk	Secret
1	X1
2	X2
3	X3
4	X4

Step 3: Hash Chains

- Each secret is hashed **multiple times**
- Example:
 - $X \rightarrow \text{hash}(X) \rightarrow \text{hash}^2(X) \rightarrow \text{hash}^3(X)$
- This is called a **hash chain**

Public Key

- For each secret:
- Hash it **maximum times (w-1)**
- If $w = 4 \rightarrow \text{max} = 3$
- So: **Public key** = $\text{hash}^3(X1), \text{hash}^3(X2), \dots$

Step 4: Signing

- From earlier:
- Message chunks = 2, 2, 3, 0
- Now:

Chunk value	What to send
2	$\text{hash}^2(X1)$
2	$\text{hash}^2(X2)$
3	$\text{hash}^3(X3)$
0	$X4$

- Signature = partially hashed secrets

Step 5: Verification

- Receiver knows:
 - Public key = max hashed values
 - Signature = partially hashed values

Example:

Chunk 1 = 2

- You sent: $\text{hash}^2(X1)$
- $\text{Max} = 3$
- Receiver does:
- $\text{hash}^2(X1) \rightarrow \text{hash}^3(X1)$
- Compare with public key

Chunk 4 = 0

- You sent: $X4$
- Friend does:
- $X4 \rightarrow \text{hash} \rightarrow \text{hash}^2 \rightarrow \text{hash}^3$
- Compare with public key
- **If all match $\rightarrow$ VALID**

Blind Signature

- A **blind signature** is a cryptographic technique in which a signer signs a message **without knowing its actual content**. This ensures the **privacy of the message**.
- **Key Points:**
 - **Privacy:** The signer cannot see or learn the message being signed.
 - **Blinding:** The message is hidden using a random value (blinding factor) before sending it to the signer.
 - **Signing:** The signer signs the blinded message.
 - **Unblinding:** The user removes the blinding factor to obtain a valid signature on the original message.

Steps in Blind Signature

- **Blinding:**
The requester (Alice) hides the message using a random **blinding factor**, so the signer cannot see its content.
- **Signing:**
The signer (Bob) signs the **blinded message** using their private key, without knowing the original message.
- **Unblinding:**
Alice removes the blinding factor to obtain a **valid signature on the original message**.

Use Cases

- **Electronic Voting:** Ensures votes remain private while being verified.
- **Digital Cash:** Allows secure transactions without revealing user identity or coin details.

Advantages of Blind Signatures

- **Privacy Preservation:** Signer cannot see the message content.
- **Non-repudiation:** Signer cannot deny the signature later.
- **Integrity:** Any change in the message invalidates the signature.

Challenges / Considerations

- **Blinding Factor Security:** Must be random and secure to prevent misuse.
- **Non-reusability:** Should not be reused for multiple messages.
- **Implementation Complexity:** Requires careful and secure implementation.

A soft, pink watercolor splash or blotch serves as a background for the text. It has irregular, feathered edges and contains subtle variations in tone, from light pink to a slightly deeper shade. Small, white, bubble-like or speckled details are scattered throughout the pink area, giving it a delicate, ethereal feel.

Thank
You